



KiCad Python API & Action Plugins

Scripting KiCad — from BOM exports to panel automation

Python automation that scales hardware engineering

Audience: Engineers comfortable with Python

API basics • Scripting console • Action plugins • Common automations



Why Script KiCad

BOM exports

Custom BOM formats
per vendor / customer.

Bulk renaming

Rename 1000 components
or 100 footprints in seconds.

Panel generation

KiKit / custom panel
builders.

Design rule checks

Custom DRCs beyond
built-in capability.

CI/CD

Headless build of
Gerbers + reports.

Reverse engineer

Read a .kicad_pcb
for analysis.

API Overview

- ▶ KiCad embeds Python interpreter (3.9+ in KiCad 8/9)
- ▶ Module: 'pcbnew' for PCB operations, 'pcbnew_swig' bindings
- ▶ Module: 'kipy' (newer, more Pythonic) introduced in KiCad 9
- ▶ Access from: built-in scripting console (Tools → Scripting Console)
- ▶ Run standalone scripts: `kicad-cli python <script>.py` (KiCad 8+)
- ▶ Action plugins: registered scripts callable from menus
- ▶ Most APIs are well-documented at docs.kicad.org

Scripting Console Basics

- ▶ Open PCB editor → Tools → Scripting Console
- ▶ Python REPL opens — full Python 3 available
- ▶ Import: `'import pcbnew'` to get the API
- ▶ Get current board: `'board = pcbnew.GetBoard()'`
- ▶ Iterate footprints: `'for fp in board.GetFootprints(): print(fp.GetReference())'`
- ▶ Iterate nets: `'for net in board.GetNetsByName().values()'`
- ▶ Iterate tracks: `'for track in board.GetTracks()'`
- ▶ Use this for one-off analyses without writing full scripts

Common API Operations

```
# Get the current board
board = pcbnew.GetBoard()

# List every footprint
for fp in board.GetFootprints():
    print(fp.GetReference(), fp.GetValue(), fp.GetPosition())

# Change a component value (e.g., rename R47 to R47A)
for fp in board.GetFootprints():
    if fp.GetReference() == "R47":
        fp.SetReference("R47A")

# Export Gerbers
plot_controller = pcbnew.PLOT_CONTROLLER(board)
plot_options = plot_controller.GetPlotOptions()
plot_options.SetOutputDirectory("gerber/")
plot_controller.SetLayer(pcbnew.F_Cu)
plot_controller.OpenPlotfile("F.Cu", pcbnew.PLOT_FORMAT_GERBER, "Top Copper")
plot_controller.PlotLayer()
plot_controller.ClosePlot()

# Save
board.Save("modified_board.kicad_pcb")
```

Action Plugins

- ▶ Plugins appear as menu items in KiCad PCB editor
- ▶ Tools → External Plugins → 'Your Plugin'
- ▶ Plugin = Python class that extends `pcbnew.ActionPlugin`
- ▶ `defcategory()`: assigns to a menu group
- ▶ `defaults()`: plugin metadata (name, description)
- ▶ `Run()`: the actual code that executes
- ▶ Install plugins: copy to `~/.config/kicad/.../scripting/plugins/`
- ▶ Most plugins distributed via KiCad Content Manager (Tools → Plugin and Content Manager)

Popular KiCad Plugins

KiKit

Panelization automation
Gerber generation

KiBOM

Custom BOM exports
Many output formats

KiBuzzard

Smart text labels
QR codes, special chars

InteractiveHtmlBom

Web-page BOM
Clickable component map

Replicate Layout

Repeat schematic blocks
as PCB sub-layouts

RoundTracks

Curve sharp corners
for better SI

Simple Plugin Template

```
import pcbnew

class HelloWorldPlugin(pcbnew.ActionPlugin):
    def defaults(self):
        self.name = "Hello World"
        self.category = "My Plugins"
        self.description = "Says hello – proves Python plugins work"
        self.show_toolbar_button = True

    def Run(self):
        board = pcbnew.GetBoard()
        n_footprints = len(board.GetFootprints())
        pcbnew.wxLogMessage(
            f"Hello! This board has {n_footprints} footprints."
        )

HelloWorldPlugin().register()
```

Common Automations

- ▶ Bulk re-route: change all 8-mil traces to 10-mil
- ▶ Component reference cleanup (R1, R2, R3 → consecutive)
- ▶ Footprint replacement (swap 0805 → 0603 across the board)
- ▶ Add stitching vias around a copper pour
- ▶ Export BOM grouped by manufacturer / package / value
- ▶ Generate fab notes PDF with stackup + impedance specs
- ▶ Move components to a new location based on schematic positions
- ▶ Color-code traces by net class for documentation

KiCad 9 'kipy' (Modern API)

- ▶ KiCad 9 introduces 'kipy' — Pythonic API replacement for SWIG bindings
- ▶ Cleaner syntax, type hints, better documentation
- ▶ Backward-compatible: existing pcbnew scripts still work
- ▶ kipy uses property-style access: 'fp.reference' vs 'fp.GetReference()'
- ▶ Modern KiCad plugins should target kipy where possible
- ▶ Documentation in progress at docs.kicad.org/kipy
- ▶ Larger community moving to kipy as KiCad 9 adoption grows

CLI for Automation

- ▶ kicad-cli: built-in CLI tool (KiCad 8+)
- ▶ Export Gerbers: 'kicad-cli pcb export gerbers <board.kicad_pcb>'
- ▶ Export PDF schematic: 'kicad-cli sch export pdf <schematic.kicad_sch>'
- ▶ Export 3D STEP: 'kicad-cli pcb export step <board.kicad_pcb>'
- ▶ Run Python script: 'kicad-cli python <script>.py' (with KiCad context)
- ▶ Useful for CI/CD pipelines — covered in deck 24
- ▶ Headless: no GUI required — runs in Docker / GitHub Actions

Common Plugin Mistakes

- ▶ Not calling `board.Save()` — changes lost
- ▶ Forgetting to handle KiCad coordinate system (mm vs nm internal)
- ▶ Modifying board while iterating over its items — undefined behavior
- ▶ Hard-coded paths — break on other systems
- ▶ Not handling `None` returns (e.g., footprint without value)
- ▶ Plugin crashes silently → check KiCad's log for error messages
- ▶ Using outdated API calls — KiCad 7→8→9 has minor breaking changes
- ▶ Not testing with a backup project — test scripts can destroy data

Plugin Distribution

- ▶ Personal use: copy script to `~/.config/kicad/<version>/scripting/plugins/`
- ▶ Team / company use: maintain in Git repo, install via symlink
- ▶ Public release: KiCad PCM (Plugin and Content Manager)
- ▶ Publish: create metadata.json, package as ZIP, submit PR to PCM repo
- ▶ PCM users install with one click from inside KiCad
- ▶ Open-source: GitHub repo + README + license (CERN-OHL or MIT)
- ▶ Examples: KiKit (most-installed PCM plugin)

Key Takeaways

Key takeaways from this lecture

API in 'pcbnew' module

KiCad embedded Python.

Scripting Console for exploration

Built-in REPL.

Action Plugins for menus

Extend KiCad GUI.

KiCad 9 'kipy'

Modern Pythonic API.

kicad-cli for CI/CD

Headless automation.

Share via PCM

Plugin distribution channel.

Questions & Discussion

Ask anything — concepts, projects, sourcing, career.

References & Further Learning

- ▶ docs.kicad.org/scripting — official KiCad Python API docs
- ▶ KiCad PCM (Plugin and Content Manager) — for plugin distribution
- ▶ KiKit on GitHub — exemplary plugin source
- ▶ kicad-cli reference (KiCad 8+)
- ▶ INTI-CMNB KiBot project — CI automation
- ▶ Robert Feranec / Phil's Lab YouTube — plugin walkthroughs
- ▶ KiCad Forum + Reddit r/KiCad
- ▶ KiCad source code on GitLab (open source = read it)

Thank you • Build something this week.